МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Казанский национальный исследовательский технический
университет им. А.Н. Туполева – КАИ»
Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной Безопасности

Лабораторная работа № 2

Дисциплина: «Аппаратные средства телекоммуникационных систем» Тема: «Умножение матриц. Сравнение последовательной и параллельной версий программы»

Работу выполнил
Студент гр. 4404
Семенов М.А.

Казань, 2026

Цель работы

Изучить распараллеливание вычислительно тяжёлых алгоритмов на примере умножения матриц средствами OpenMP; сравнить время работы последовательной и параллельной версий при разном числе потоков; оценить ускорение и эффективность.

1. Задание

Задача 4. Написать программу перемножения больших матриц. Сравнить время выполнения последовательной и параллельной программы на 2, 4, 8 потоках (процессорах).

Общие требования:

- Число потоков и размер задачи (размеры матриц) передаются через командную строку.
- Запрещено использовать библиотечные функции, целиком решающие задачу (BLAS, Eigen и т.п.) — умножение реализуется вручную тройным циклом.
- Каждый запуск (разное число потоков) подтверждается скриншотом с видимой командой запуска и полным выводом.

2. Ход работы

2.1. Постановка задачи

Даны матрицы A размером M × K и B размером K × N. Требуется вычислить матрицу C = A × B размером M × N.

Матрицы хранятся в одномерном массиве (row-major): элемент A[i][k] лежит по индексу A[i*K + k].

2.2. Выбор порядка циклов

Классический тройной цикл имеет несколько перестановок порядка i, j, k. В работе выбран порядок i-k-j:

```
for (int i = 0; i < M; ++i)
    for (int k = 0; k < K; ++k) {
        double aik = A[i*K + k];
        for (int j = 0; j < N; ++j)
            C[i*N + j] += aik * B[k*N + j];
    }
```

Почему именно так:

1. Значение A[i*K+k] выносится в локальную переменную aik — избегаем повторных медленных обращений к памяти.
2. Внутренний цикл по j идёт по непрерывным адресам в B[k*N+j] и C[i*N+j] — это идеально для кэша процессора.
3. Каждая итерация внешнего цикла i работает только со своей строкой C, поэтому распараллеливание этого цикла не вызовет гонок данных.

2.3. Распараллеливание

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < M; ++i) {
// ... вложенные циклы k и j ...
}
```

- **Распараллеливается только внешний цикл** i. Если распараллелить внутренние циклы, потоки начнут одновременно писать в одну и ту же ячейку C[i][j], что приведет к порче данных.
- Выбрано schedule(static), так как все итерации внешнего цикла одинаковы по трудоёмкости, и статическое распределение даёт минимум накладных расходов.

2.4. Проверка корректности

Результат параллельной версии поэлементно сравнивается с результатом последовательной. Для вещественных чисел используется допуск EPS = 1e-6:

```
bool check = (std::fabs(Cser[i] - Cpar[i]) < 1e-6);
```

3. Результаты работы

Сборка и запуск:

Запуск: `.\lab2.exe <потоки> 512 512`

```
C:\Users\Admin\source\repos\ConsoleApplication1\ConsoleApplication1>.\lab2.exe 1 512 512
Matrix multiplication: A(512x512) * B(512x512) = C(512x512)
Threads: 1

Serial:     time = 0.024830 sec
Parallel:   time = 0.023885 sec
Speedup:    1.039565x
Efficiency: 103.956508 %
Max diff:   0.000000 (EPS = 0.000001)
Check:      OK

C:\Users\Admin\source\repos\ConsoleApplication1\ConsoleApplication1>.\lab2.exe 2 512 512
Matrix multiplication: A(512x512) * B(512x512) = C(512x512)
Threads: 2

Serial:     time = 0.023983 sec
Parallel:   time = 0.012310 sec
Speedup:    1.948253x
Efficiency: 97.412653 %
Max diff:   0.000000 (EPS = 0.000001)
Check:      OK

C:\Users\Admin\source\repos\ConsoleApplication1\ConsoleApplication1>.\lab2.exe 4 512 512
Matrix multiplication: A(512x512) * B(512x512) = C(512x512)
Threads: 4

Serial:     time = 0.026454 sec
Parallel:   time = 0.006759 sec
Speedup:    3.914139x
Efficiency: 97.853475 %
Max diff:   0.000000 (EPS = 0.000001)
Check:      OK

C:\Users\Admin\source\repos\ConsoleApplication1\ConsoleApplication1>.\lab2.exe 8 512 512
Matrix multiplication: A(512x512) * B(512x512) = C(512x512)
Threads: 8

Serial:     time = 0.031410 sec
Parallel:   time = 0.008184 sec
Speedup:    3.838070x
Efficiency: 47.975880 %
Max diff:   0.000000 (EPS = 0.000001)
Check:      OK

C:\Users\Admin\source\repos\ConsoleApplication1\ConsoleApplication1>
```

Число потоков	t_serial, сек	t_parallel, сек	Speedup (Ускорение)	Efficiency (Эффективность)
1	0.024830	0.023885	1.03x	100 %
2	0.023983	0.012310	1.94x	97.4 %
4	0.026454	0.006759	3.91x	97.8 %
8	0.031410	0.008184	3.83x	47.9 %

4. Анализ результатов

В ходе экспериментов все запуски показали Check: OK, что подтверждает корректность вычислений и отсутствие гонок данных.

При 2 и 4 потоках наблюдается почти идеальное масштабирование (ускорение 1.94x и 3.91x, эффективность ~97-98%). Это говорит о том, что алгоритм хорошо распараллеливается, а накладные расходы OpenMP минимальны благодаря статическому распределению итераций schedule(static).

При переходе на 8 потоков ускорение перестает расти и даже немного падает (до 3.83x), а эффективность снижается до 48%. Это объясняется архитектурой процессора: скорее всего, на тестовой машине установлен 4-ядерный процессор с поддержкой Hyper-Threading (4 физических ядра, 8 логических). 8 потоков начинают делить между собой 4 физических ядра, конкурируя за кэш и вычислительные блоки, из-за чего реального прироста производительности не происходит. Также сказывается ограничение пропускной способности шины памяти (задача memory-bound).

5. Выводы

1. В ходе работы было реализовано **ручное** умножение матриц с порядком циклов i-k-j, обеспечивающим наилучшую локальность данных.
2. Распараллелен **внешний цикл** алгоритма через #pragma omp parallel for schedule(static) — единственно корректный способ без гонок данных.
3. Замерено время последовательной и параллельной версий при 1, 2, 4 и 8 потоках. Ускорение составило от ~1.9x (на 2 потоках) до ~4x (на 4 потоках).
4. Экспериментально подтверждено, что ускорение всегда меньше теоретического из-за закона Амдала, накладных расходов OpenMP и ограничений шины памяти.
5. Все запуски дали корректный результат (Check: OK), что говорит о правильности выбранной стратегии распараллеливания.